

## 第 2 章：HTTP 協定 (HyperText Transfer Protocol)

HTTP 是網際網路的基石，它定義了客戶端（例如你的瀏覽器）與伺服器之間交換資料的規則。理解 HTTP 是開發 Web 應用程式、API 以及進行網路排錯的最核心職責。

---

[nlh Slide](#)

- Web and HTTP
  - History of HTTP
  - HTTP
    - stateless and keep-alive
    - 3 way handshake
- 

### 2.1 HTTP 基本概念

HTTP 是一種基於文本的協議。作為資工系學生，我們通常將它視為客戶端和伺服器之間通訊的主要協議。

#### 2.1.1. HTTP 是架構在 TCP 之上

HTTP 協定主要運作於傳輸層的 **TCP (Transmission Control Protocol)** 之上（HTTP/3 則轉向 UDP/QUIC）。

- **TCP 的核心特色：**
  - **\*\*可靠性 (Reliability)\*\***：透過序號與確認應答 (ACK) 機制，確保數據完整到達。
  - **\*\*三向交握 (Three-way Handshake)\*\***：建立連接前先確認雙方都有收發能力 (SYN, SYN-ACK, ACK)。
  - **\*\*有序傳輸 (Ordered Delivery)\*\***：保證數據包按發送順序組裝。
  - **\*\*流量控制 (Flow Control)\*\***：防止發送端速度過快導致接收端緩衝區溢位。
- **HTTP 繼承的優勢**：因為 TCP 是可靠的，Web 開發者在撰寫 HTTP 請求時，不需要擔心封包遺失或內容亂序，TCP 會在底層自動處理這些複雜的網路問題。

#### 2.1.2 HTTP 發展歷史 🕒

HTTP 的演進反映了 Web 從簡單文件分享到複雜應用平台的過程：

- **\*\*HTTP/0.9 (1991)\*\***：由 **Tim Berners-Lee** 在 CERN（歐洲核子研究組織）發明。
  - **創立緣由**：當時科學家們使用的電腦系統各異，資訊互不相通。Tim 為了讓全世界的教育與研究機構能跨平台分享文件，設計了這一套全球資訊網 (WWW) 的基礎架構。
- **\*\*HTTP/1.0 (1996)\*\***：新增了標頭 (Headers)、狀態碼與多媒體支援 (MIME types)。
- **\*\*HTTP/1.1 (1997)\*\***：目前最普及的版本。引入了 **持久連接 (Keep-Alive)** 與快取管理，顯著提升效能。
- **HTTP/2 (2015)**：效能大躍進。改為二進位框架，支援**\*\*多路復用 (Multiplexing)\*\***，解決了連線阻塞問題。
- **\*\*HTTP/3 (2022)\*\***：基於 UDP 的 QUIC 協議。進一步減少連接建立延遲，並加強了網路切換（如從 Wi-Fi 切換到 5G）時的穩定性。

- **目前普及狀況**：雖然 HTTP/3 是最先進的版本且被 Google, Facebook 等大型服務廣泛採用，但 **HTTP/2 與 HTTP/1.1 目前仍是 Web 上最普遍通用的版本**。HTTP/3 的普及依賴於瀏覽器與伺服器端的硬體優化與協定支援的完善。

### Keep-Alive 機制

在使用 HTTP/1.0 時，瀏覽器每請求一個檔案（如一張圖片）就必須建立一次 TCP 連線，請求完就斷開。這會導致大量的效能損耗（因為要不斷重複「三次握手」）。

- **定義**：在 HTTP/1.1 中預設啟用的 **\*\*持久連接 (Persistent Connection)\*\***。
- **好處**：讓多個 HTTP 請求/回應可以重複使用同一個已建立好的 TCP 連線，大幅減少了建立連線的時間延遲。

### 2.1.3 HTTP 方法 (HTTP Methods)

HTTP 協議定義了多種方法來與伺服器進行交互，最常見的有：

- **GET**: 用於請求資源，通常用來讀取資料。
- **POST**: 用於提交資料，通常用來創建或修改伺服器上的資料。
- **PUT**: 用於更新資料。
- **DELETE**: 用於刪除資料。
- **HEAD**: 像 GET 方法，但不返回消息體，僅返回標頭。
- **PATCH**: 用於對資料進行部分修改。

[!TIP] 💡 **重點觀念**：HTTP 是一種「請求-回應 (Request-Response)」模式。除非客戶端發起請求，否則伺服器通常不會主動傳送資料給客戶端。

[!WARNING] ⚠️ **小提示**：GET 請求的參數會顯示在網址 (URL) 中，因此不適合用來傳輸密碼等敏感資訊。雖然 POST 相對安全，但若不配合 HTTPS，資料仍可能被截獲。

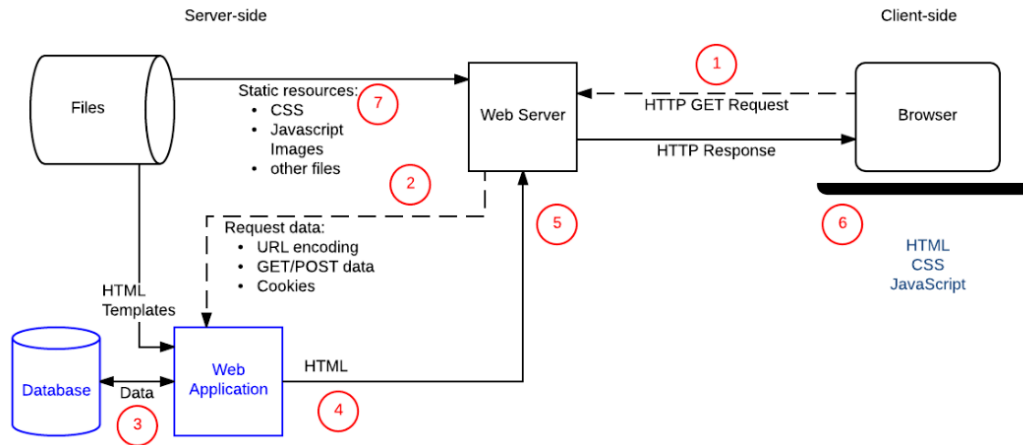
### 2.1.4 HTTP 請求的生命週期

當你在瀏覽器輸入網址後，會經歷以下階段：

1. **DNS 查詢**: 將域名轉換為 IP 地址。
2. **TCP 連接**: 三次握手 (SYN -> SYN-ACK -> ACK)。
3. **TLS 握手** (如果是 HTTPS): 密鑰交換。
4. **傳送請求**: 客戶端發送 HTTP Data。

5. 伺服器處理: 程式碼運算與資料庫查詢。

## Server Side Programming



6. 傳送回應: 伺服器回傳資料。

7. 瀏覽器渲染: 解析 HTML, CSS, JS 並顯示頁面。

### 2.1.5 HTTPS 與傳輸安全

- **HTTP:** 不加密，數據以純文本傳輸，易被竊聽。
- **HTTPS:** 使用 **TLS/SSL** 加密，保護隱私與完整性。其核心機制包含：
  - **\*\*數位憑證 (Digital Certificate)\*\*:** 伺服器的「數位身分證」，由公信機構 (**CA**) 簽發。內容包含伺服器的 **\*\*公鑰 (Public Key)\*\***，確保你連上的是真正的網站而非冒牌貨。
  - **\*\*確保隱私 (Privacy)\*\*:**
    1. **金鑰交換:** 利用憑證中的公鑰進行非對稱加密，安全地協商出一個後續通訊用的臨時金鑰。
    2. **資料加密:** 雙方改用該臨時金鑰進行對稱加密，將數據轉換為亂碼。即使被截獲，駭客也無法解讀原始內容。
  - **\*\*確保完整性 (Integrity): 使用數位簽章與雜湊 (Hashing)\*\*:** 瀏覽器會比對收到的數據雜湊值與簽章是否相符，若有「中間人」修改過哪怕一個位元組，瀏覽器都會立即彈出不安全警告。

## 2.2 HTTP 請求與回應結構

當我們在瀏覽器打上一段網址，例如 `https://www.example.com:443/person/123?name=John&age=25`，然後按下 Enter，這時瀏覽器就會向伺服器發送一個 HTTP 請求。

網址 (URL) 組成拆解：

- **https:** **\*\*通訊協定 (Protocol)\*\***。定義瀏覽器與伺服器溝通的方式。
- **www.example.com:** 網域位址 (Domain Name/Host)。伺服器在網路上的身分地址。
- **:443:** **\*\*連接埠 (Port)\*\***。類似大樓的「分機號碼」或「門牌號碼」。伺服器上可以跑很多服務，Port 確保請求送達正確的程式 (https 預設為 443，http 預設為 80)。
- **/person/123:** **\*\*資源路徑 (Path)\*\***。告訴伺服器你具體想要哪一個資源。

- **?name=John&age=25**：**\*\*查詢參數 (Query Parameters)\*\***。用來傳遞額外的資料給伺服器，多個參數間用 `&` 隔開。

### 2.2.1 HTTP 請求 (Request)

當客戶端向伺服器發送請求時，它會包含以下內容。

範例：一個典型的 HTTP GET 請求

```
GET /course/ch02_http.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0 (Macintosh)
Accept: text/html
```

保留字意義說明：

- **GET**：**\*\*方法 (Method)\*\***。告訴伺服器此請求的操作類型（此處為「索取資源」）。
- **/course/ch02\_http.html**：**\*\*路徑 (Path/Resource)\*\***。指定要存取的資源具體位置。
- **HTTP/1.1**：**\*\*協定版本 (Protocol Version)\*\***。告訴伺服器應使用哪一個版本的 HTTP 規則進行溝通。
- **Host**：**\*\*標頭欄位-主機 (Header: Host)\*\***。必填欄位，指明請求的目標域名（這讓一台伺服器可以同時代管多個網站）。
- **User-Agent**：**使用者代理**。描述發起請求的瀏覽器與作業系統資訊。

*[!TIP] 為什麼近代瀏覽器都自稱 **Mozilla**？這是網頁開發史上的一段「偽裝史」。早期 Netscape (代號 Mozilla) 支援框架 (Frames)，而其他瀏覽器不支援。伺服器會檢查 User-Agent，若不是 Mozilla 就傳送沒框架的簡易版。為了能正確顯示網頁，IE、Chrome、Safari 等瀏覽器後來都紛紛在 User-Agent 開頭加上 Mozilla/X.X 以「欺騙」伺服器它們也是相容的瀏覽器。這就是為什麼現在即便你用 Chrome，標頭仍可能看到 `Mozilla/5.0...Chrome/...`。*

### 2.2.2 HTTP 回應 (Response)

伺服器會返回回應，通常包括以下內容。

範例：一個典型的 HTTP 成功回應

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1024
Connection: keep-alive

<!DOCTYPE html>
<html lang="zh-Hant">
<head>
  <meta charset="UTF-8">
  <title>Hello World</title>
</head>
<body>
  <h1>你好，Web 開發者！</h1>
```

```
<p>這是一個從伺服器回傳的真實 HTML 內容。</p>
</body>
</html>
```

保留字意義說明：

- **HTTP/1.1**：協定版本。確認雙方互動使用的通訊標準。
- **200**：\*\*狀態碼 (Status Code)\*\*。以三位數字代表處理結果（200 代表「成功」）。
- **OK**：\*\*狀態訊息 (Reason Phrase)\*\*。對狀態碼的人類可讀簡短描述。
- **Content-Type**：內容類型。告訴瀏覽器回傳的資料性質（如 HTML 文本、圖片或 JSON），以便瀏覽器正確渲染。
- **Content-Length**：內容長度。以位元組 (Bytes) 為單位，指明回應正文的大小。

---

### 2.2.3 HTTP 狀態碼 (Status Codes)

你可以透過首位數字快速判斷請求的結果：

- **1xx (Informational)**: 正在處理中。
- **2xx (Success)**: 請求成功。(例如：200 OK)
- **3xx (Redirection)**: 搬家了，請去別的地方。(例如：301 Moved Permanently)
- **4xx (Client Error)**: 你的錯！(例如：404 Not Found, 400 Bad Request)
- **5xx (Server Error)**: 我的錯 (伺服器噴錯)。(例如：500 Internal Error)

[!IMPORTANT] ⚠️ **常見面試題**：404 (Not Found) 與 500 (Internal Server Error) 的區別？404 是找不到該資源，通常是網址打錯；500 則是伺服器程式碼執行時發生錯誤 (Crash)。

### 👉 自我測驗 (Self-Test)

▶  點擊查看測驗問題與解答

---

## 2.3 實作演練 (Practical Exercises)

### DevTools 觀察員

打開瀏覽器，進入[逢甲資工系](#)首頁，按下 F12, 或是直接在頁面按滑鼠右鍵，選擇「檢查」，觀察[逢甲資工系](#)的請求 Headers。

### DevTools 常用面板介紹

DevTools 是前端開發者的「瑞士刀」，主要包含以下幾個切換分頁：

- **\*\*Elements (元素)\*\***：顯示目前網頁的 **HTML 結構** 與 **CSS 樣式**。你可以在這裡即時修改文字或顏色，觀察網頁的變化（這不會影響伺服器上的原始檔案，重新整理後就會恢復）。
- **\*\*Console (主控台)\*\***：顯示程式碼的輸出訊息或錯誤 (Logs & Errors)。你也可以在這裡直接輸入 JavaScript 程式碼並執行。
- **Network (網路)**：本章的核心工具。它記錄了所有瀏覽器發出與收到的網路傳輸請求。

如何在 **Network** 面板觀察 HTTP 請求？

1. 打開 DevTools 並切換到 **Network** 分頁。
2. **\*\*重新整理網頁 (F5)\*\***：你會看到下方列出了一長串網頁載入的資源（HTML, CSS, 圖片, API）。

3. 選擇一個資源：點擊清單中的第一筆（通常是該網頁的網域名稱），右側會彈出詳細資訊視窗：

- **Headers**：這就是我們學習的重點！
  - **General**：顯示請求網址、方法 (GET/POST) 以及狀態碼 (200 OK)。
  - **Response Headers**：伺服器發回來的標頭。
  - **Request Headers**：你的瀏覽器主動發出去的標頭。
- **\*\*Payload (負載)\*\***：如果你發送的是 POST 請求，可以在這裡看到你填寫的表單內容。
- **Preview / Response**：顯示伺服器回傳的原始資料內容（如完整的 HTML 程式碼或圖片預覽）。

## HTTP 請求的完整生命週期 (Network Timing)

當你在瀏覽器按下網址後，你可以透過以下方式查看精確的請求過程：

- **法 A：切換分頁**：在 Network 面板點擊清單中的資源（例如第一個 HTML 檔案），在右側彈出的詳細資訊面板中，找到頂部的 **Timing** 標籤（通常在 Headers, Payload, Preview 旁邊）。
- **法 B：快速預覽**：直接將滑鼠游標懸停 (**Hover**) 在清單右側的 **Waterfall** 顏色條上，就會彈出一個摘要小視窗，顯示從排隊到下載的各階段時間。

透過 Timing 分頁或 Waterfall 摘要，你可以看到：

### 1. 資源調度階段 (Resource Scheduling)

- **\*\*Queueing (排隊)\*\***：瀏覽器正在等候。原因可能是：請求優先權較低、連線數已達上限（Chrome 對單一域名最多 6 個連線）、或是正在分配磁碟快取空間。
- **\*\*Stalled (停滯)\*\***：請求可能因為等待建立 TCP 連線或等待代理伺服器 (Proxy) 處理而卡住。

### 2. 連線建立階段 (Connection Setup)

- **\*\*DNS Lookup (網域解析)\*\***：瀏覽器正在找尋該域名對應的 IP 地址。
- **\*\*Initial Connection (初始連線)\*\***：進行 TCP 的「三向交握」。
- **SSL/TLS Handshake**：如果是 HTTPS，這是雙方交換密鑰、建立加密通道的時間。

### 3. 請求與回應階段 (Request/Response)

- **\*\*Request Sent (傳送請求)\*\***：瀏覽器將 HTTP 請求封包完全發送出去的時間。
- **Waiting (TTFB - Time to First Byte)**：最關鍵的效能指標。這是從發送請求後，到收到伺服器回傳「第一個位元組」的時間。這段時間包含了「網路往返」以及「伺服器轉動程式碼、查詢資料庫」的時間。
- **\*\*Content Download (內容下載)\*\***：瀏覽器持續接收伺服器回傳資料的時間。如果檔案很大或網路很慢，這一段會拉得很長。

## 終端機大師

嘗試執行 `curl` 指令，這是一個強大的工具，讓你在不需要瀏覽器的情況下直接觀察 HTTP 封包。

### 如何打開終端機 (Terminal)？

- **Windows**：
  1. 按下 Win + R 鍵，輸入 `cmd` 或 `powershell` 並按 Enter。
  2. 在現代 Windows (10/11) 中，代號 `curl` 的工具已經內建，可以直接使用。
- **macOS**：

1. 按下 `Command + Space` (Spotlight)，搜尋 `Terminal` (終端機) 並打開。

- **Linux：**

1. 使用快捷鍵 `Ctrl + Alt + T` 打開終端視窗。

**練習指令：**

1. **\*\*觀察標頭 (Headers Only)\*\*：**

```
curl -I https://www.fcu.edu.tw
```

這會僅顯示伺服器回傳的 **Response Headers**，適合快速檢查狀態碼。

```
HTTP/2 200
content-type: text/html; charset=UTF-8
date: Wed, 04 Mar 2026 07:57:44 GMT
referrer-policy: no-referrer-when-downgrade
server: Apache/2.4.54 (Ubuntu)
link: <https://www.fcu.edu.tw/wp-json/>; rel="https://api.w.org/"
link: <https://www.fcu.edu.tw/wp-json/wp/v2/pages/2>; rel="alternate";
type="application/json"
link: <https://www.fcu.edu.tw/>; rel=shortlink
access-control-allow-headers: origin, x-requested-with, content-type
access-control-allow-methods: PUT, GET, POST, DELETE, OPTIONS
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
pragma: no-cache
cache-control: max-age=0, no-cache, no-store, must-revalidate
x-xss-protection: 1; mode=block
set-cookie: language=zh; expires=Thu, 04-Mar-2027 07:57:44 GMT; Max-Age=31536000; path=/; SameSite=Strict; Secure; HTTPOnly
strict-transport-security: max-age=31536000
x-cache: Miss from cloudfront
via: 1.1 ea260d0f1b9f37d9bfa2a38aa91766cc.cloudfront.net (CloudFront)
x-amz-cf-pop: TPE54-P2
x-amz-cf-id: N5TxjUxLehV35gh0FyNkeJD4YfJyC6WLjvcn8PkEj91Ee5sG8WdYnA==
```

**如何解讀這些標頭資料？**

從上面的 `curl -I` 結果中，我們可以看到幾項關鍵資訊：

- **HTTP/2 200**：確認連線使用 **HTTP/2** 協定，且狀態碼為 **\*\*200 (成功)\*\***。
- **content-type: text/html**：告知瀏覽器回傳的是 UTF-8 編碼的 HTML 網頁。
- **\*\*server: Apache/2.4.54 (Ubuntu)\*\***：揭露了伺服器使用的軟體版本為 Apache，跑在 Ubuntu 作業系統上。
- **set-cookie**：伺服器正在向瀏覽器寫入 **Cookie**（此處為語系設定 language=zh），這是 HTTP 處理「狀態」的常見方式。
- **\*\*strict-transport-security (HSTS)\*\***：強制瀏覽器在未來一段時間內只能透過 **HTTPS** 進行連線，確保傳輸安全。

- **x-cache: Miss from cloudfront & via**：這表示網站使用了 **CDN (CloudFront)** 加速技術。  
**Miss** 代表這是直接從原始伺服器抓取的，而非緩衝區。

2. **\*\*詳細模式 (Verbose Mode)\*\***：

```
curl -v https://www.fcu.edu.tw
```

這會完整顯示「三次握手」、「Request Headers」以及「Response Headers」，是排錯時的最佳幫手。